西南交通大學
Southwest Jiaotong University

云计算技术

第6讲 消息队列与事件驱动架构

授课教师：戴朋林

课程代码：SCAI004712

2026年4月



00 本讲内容大纲

- 消息队列基础
  - ✓ 消息队列的概念与作用
  - ✓ 消息模型：点对点（Queue）与发布/订阅（Topic）
  - ✓ 消息投递语义（At Most Once、At Least Once、Exactly Once）
  - ✓ 消息顺序性保证
- Apache Kafka
  - ✓ Kafka的架构设计（Broker、Topic、Partition、Consumer Group）
  - ✓ Kafka的日志收集与流处理应用
- RabbitMQ
  - ✓ AMQP协议基础
  - ✓ RabbitMQ核心概念（Exchange、Queue、Binding、Routing Key）
  - ✓ 消息确认机制（ACK）与持久化
  - ✓ 死信队列（DLQ）与延迟消息
- 事件驱动架构（EDA）
  - ✓ 事件驱动架构的设计理念与优势
  - ✓ 事件溯源（Event Sourcing）与CQRS模式简介
- Redis Pub/Sub与Stream
  - ✓ Redis作为轻量级消息中间件
  - ✓ Pub/Sub模式的使用场景与局限性
  - ✓ Redis Stream的持久化消息队列能力
- ✓ 应用示例与演示代码

西南交通大学
Southwest Jiaotong University

1 消息队列基础

01 为什么需要消息队列

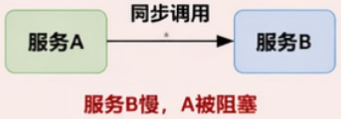
西南交通大学 Southwest Jiaotong University

- 在分布式系统中，服务之间经常需要交换数据，异步处理任务或应对流量波动。
- 消息队列作为一种异步通信中间件，能够有效提升系统的解耦性，可扩展性和可维护性。

没有消息队列时的常见问题

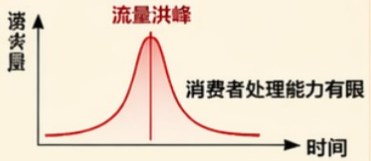
强耦合

- 生产者需要等待消费者处理完成
- 服务之间直接调用，依赖关系复杂



流量峰值导致系统崩溃

- 突发流量下，消费者来不及处理
- 请求直接堆积，最终导致服务不可用



可靠性不足

- 消息处理失败时可能丢失
- 难以实现重试和最终一致性



引入消息队列后的优势

异步解耦

生产者发送消息后无需等待，消费者异步处理，服务之间解耦

流量削峰

消息在队列中缓冲，消费者按自身能力消费，平滑应对流量洪峰

可靠传输

支持持久化、确认机制、重试等，保证消息不丢失

最终一致性

通过消息驱动异步完成操作，保证系统在一段时间后达到一致状态

01 消息队列的概念与作用

西南交通大学 Southwest Jiaotong University

- 消息队列(Message Queue, MQ) 是一种应用间通过消息进行**异步通信**的中间件。
- 核心作用
  - ✓ 异步解耦：发送方无需等待接收方处理， 降低系统耦合。
  - ✓ 流量削峰：缓冲突发流置， 保护下游服务不披压垮。
  - ✓ 最终一致性： 通过可靠消息传递，达成系统最终一致的状态。

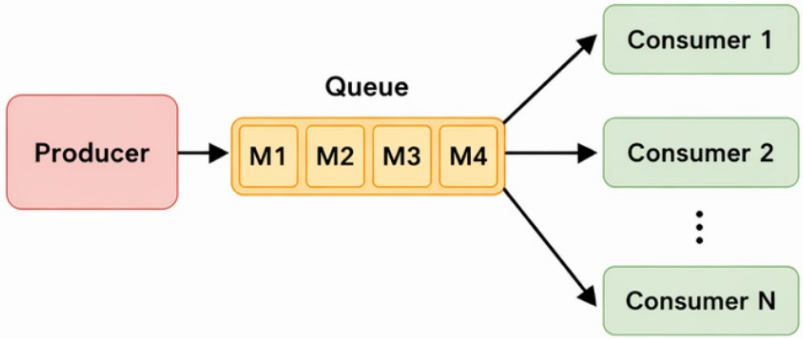


02 消息模型：点对点（Queue）与发布/订阅（Topic）

西南交通大学 Southwest Jiaotong University

点对点（Queue）

- 消息发送到队列，只有一个消费者能消费。
- 适用于任务分发、负载均衡。

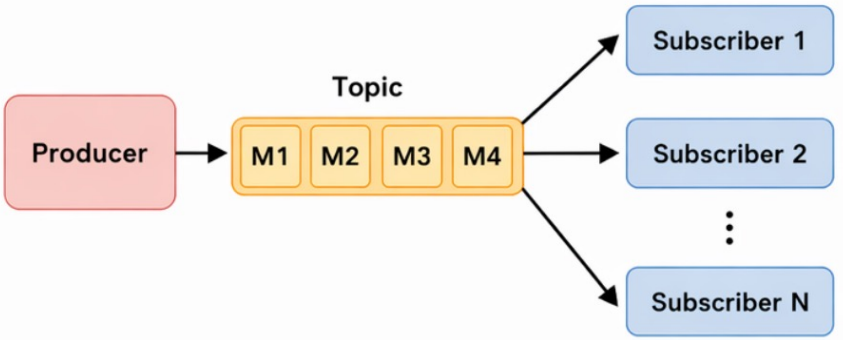


特点

- 每条消息只会被一个消费者消费
- 消费者竞争消费，实现负载均衡
- 不适合广播场景

发布/订阅（Topic）

- 消息发布到主题，所有订阅者都能收到。
- 适用于广播通知、事件分发。



特点

- 每条消息会被所有订阅者消费
- 支持一对多广播
- 适合事件通知、日志分发等场景

03 消息投递语义：At Most Once（最多一次）

西南交通大学 Southwest Jiaotong University

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

At Most Once

- 每条消息最多投递一次，即消息可能会丢失，但绝不会重复发送。
- 适用场景：
  - 消息丢失不影响系统的正常运行，系统能容忍消息丢失，例如日志数据收集（丢失一些日志不影响整体分析结果）。
  - 适用于对消息可靠性要求不高的场景。
- 优点：效率高，处理过程简单，消息丢失时不需要额外的重试机制。
- 缺点：由于消息丢失的可能性，不能保证消息的完整性。

03 消息投递语义：At Least Once（至少一次）

西南交通大学 Southwest Jiaotong University

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

At Least Once

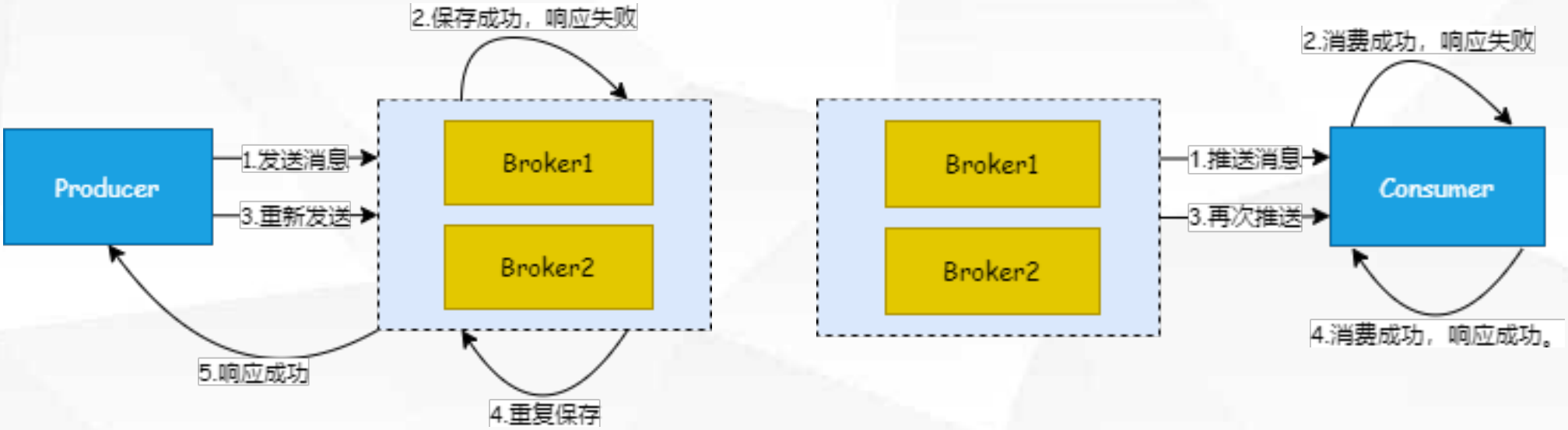
- 每条消息至少投递一次，保证每个消息至少有一个消费者接收。
- 适用场景：
  - 系统中非常注重**数据完整性**，**重复消费**并不会带来严重问题的场景。
  - 例如订单支付系统，如果多次支付请求被处理，最终会给用户产生冗余的扣款，但不会丢失任何支付请求。
- 优点：消息不会丢失，适用于对消息可靠性有较高要求的场景。
- 缺点：可能会有消息**重复投递**的情况，需要消费者能够处理重复数据，例如幂等性。

03 消息投递语义：At Least Once（至少一次）

西南交通大学 Southwest Jiaotong University

什么情况下会导致消息重复?

- 生产者发送消息后，Broker 保存成功，但是没有成功给生产者返回 ACK
- 消费者消费消息后，给 Broker 返回 ACK 失败，导致 Broker 没有修改偏移量，同一条消息再次发送



03 消息投递语义：Exactly Once（恰好一次）

西南交通大学 Southwest Jiaotong University

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

Exactly Once

- 每条消息仅投递一次，并且没有重复消费的情况，确保消息只会被处理一次。
- 适用场景：
  - 对消息的准确性和重复消费非常敏感的场景，如银行转账、订单交易等。
  - 例如支付系统中，若订单支付成功，不能再重复扣款。
- 优点：完美的消息传递保证，确保消息只会被消费一次，适用于高价值和高准确性的场景。
- 缺点：实现复杂，性能开销较大，通常需要高级的幂等性保证和事务机制。

03 消息投递语义

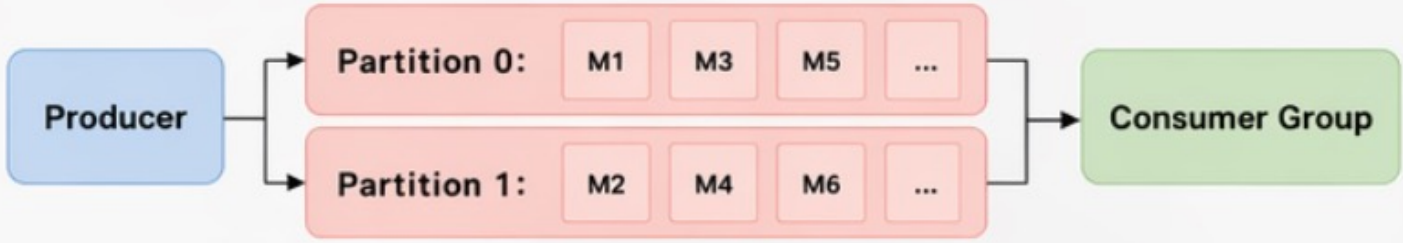
西南交通大学 Southwest Jiaotong University

投递语义	适用场景	优点	缺点
At Most Once	性能要求高、对消息丢失不敏感（如日志收集、事件记录）	高效，处理过程简单，性能开销低	可能会丢失消息，无法确保消息完整性
At Least Once	数据完整性要求较高，重复处理不影响（如订单支付、社交媒体消息）	不会丢失消息，保证消息传递可靠	可能会导致消息重复，需要额外处理重复数据（幂等性）
Exactly Once	高价值交易、金融支付、库存管理等高敏感场景	完美的消息传递保证，消息准确且无重复消费	实现复杂，性能开销大，通常需要高级的幂等性保证

04 消息顺序性保证

西南交通大学 Southwest Jiaotong University

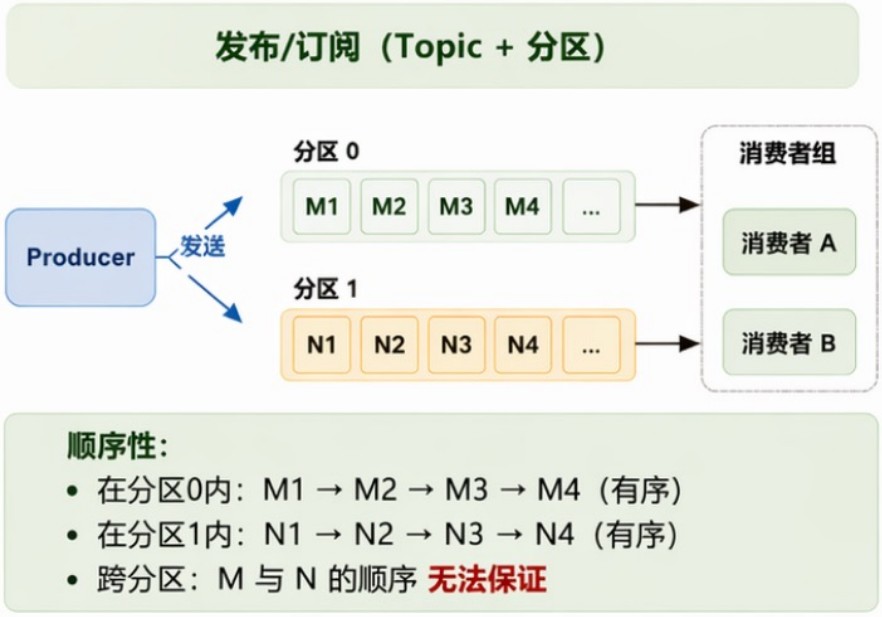
- **消息顺序性**是指在消息传递过程中，消息的接收顺序与发送顺序相同。
- 对于一些需要强一致性、高可靠性的系统（如交易系统），消息顺序性的保证是核心需求。
- 顺序性常见的保证方式：
  - 单分区保证： 在同一分区内，消息是有序的(Kafka、Redis Stream)
  - 单队列保证： 在同一队列中，消息具有FIFO （ 先入先出）
  - 多分区/多队列：无法保证全局顺序，只保证分区 / 队列内顺序



增加消息队列数量/分区数量，可增加并行性，也会牺牲全局顺序性

04 消息顺序性保证

特性	队列（Queue）	分区（Partition）
消息顺序	严格按照发送顺序消费	只保证单个分区内的顺序，跨分区不保证顺序
适用模型	点对点（Queue）	发布/订阅（Topic）
消费者	每条消息只被一个消费者消费	每个分区有独立消费者，多个消费者并行消费不同分区
并行性	不支持并行消费	支持并行消费，不同消费者处理不同分区的数据



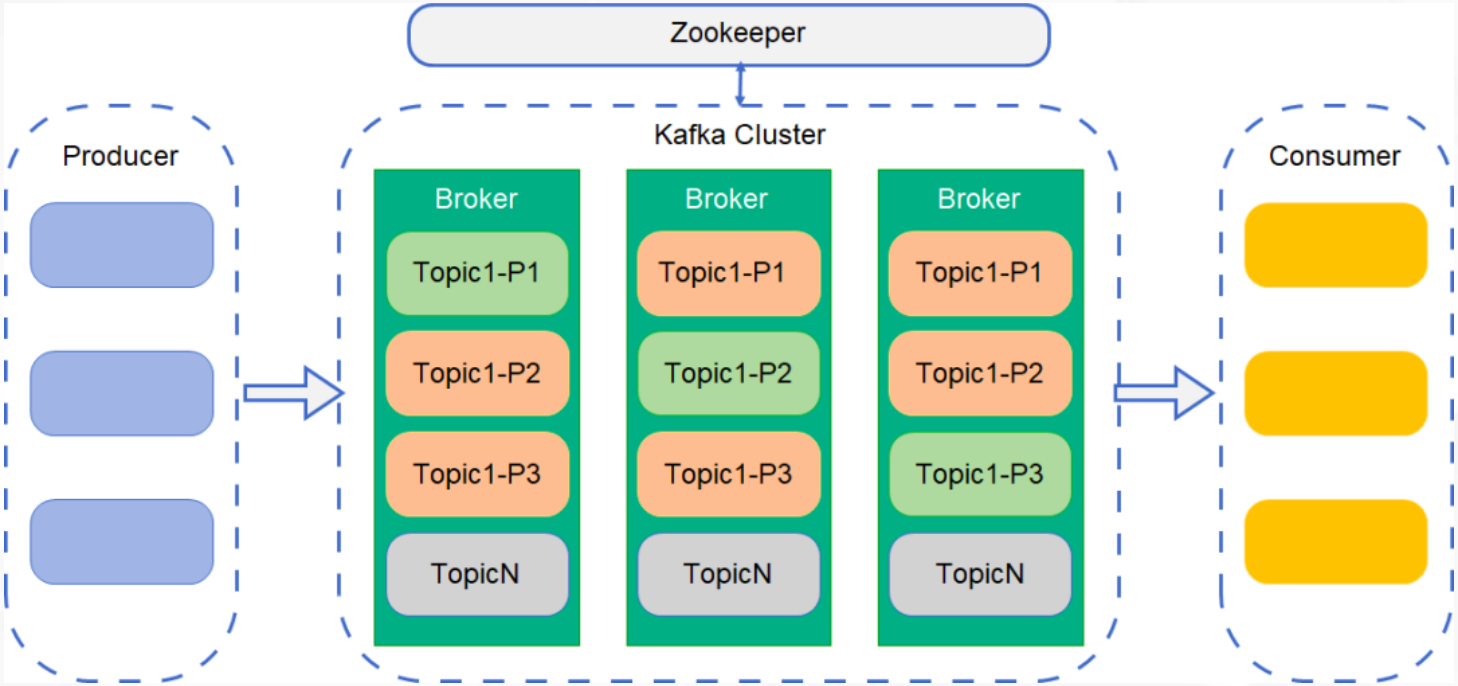


西南交通大学
Southwest Jiaotong University

2 Apache Kafka

01 Kafka基本介绍

- Kafka是最初由Linkedin公司开发，是一个分布式的发布/订阅消息系统
- Linkedin于2010年贡献给了Apache基金会并成为顶级开源项目
- Kafka核心由Broker、Topic、Partition、Consumer Group组成



APACHE kafka
A distributed streaming platform

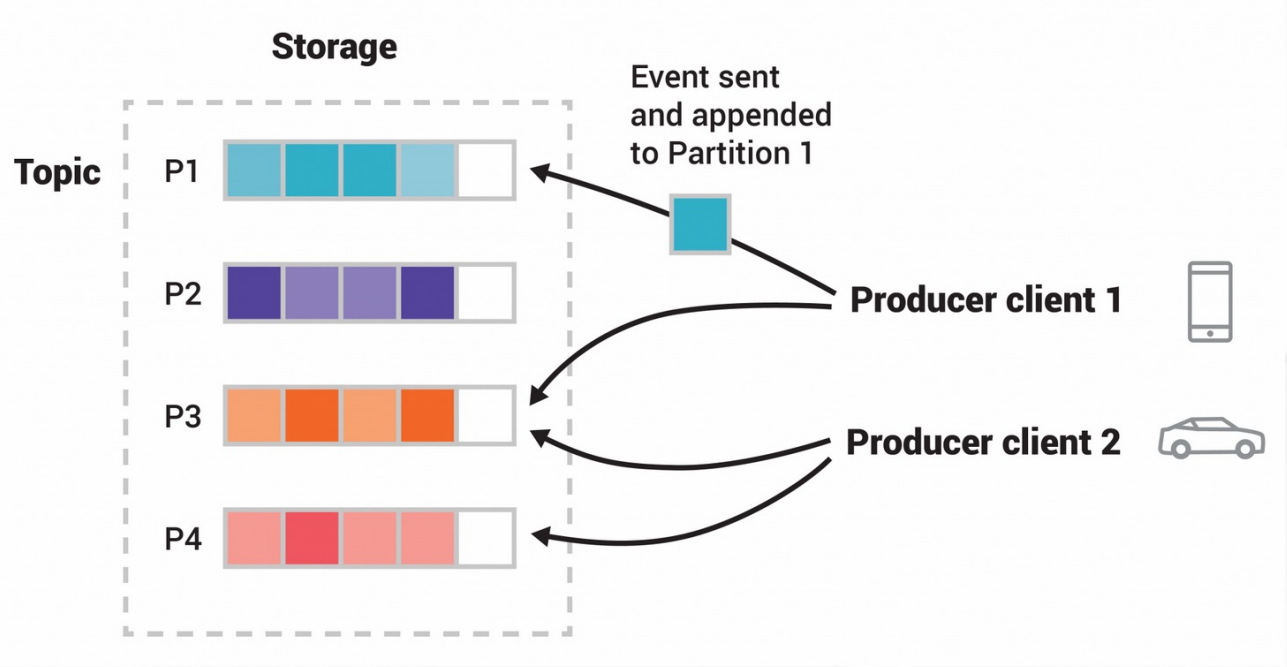
02 Kafka基本架构组件

西南交通大学 Southwest Jiaotong University

组件	说明	关键特性
Broker	Kafka 集群中的服务器节点，负责数据存储，请求处理和副本管理。	- 水平扩展 - 高可用（副本机制）
Topic	消息的逻辑标识，用于分类和隔离数据。	- 支持多个分区 - 可配置副本数
Partition	Topic 的物理分片，是 Kafka 并行处理和扩展的基本单位。	- 分区内消息有序 - 分区间可并行
Consumer Group	一组消费者实例，共同消费一个或多个 Topic。	- 组内负载均衡 - 组间互不影响

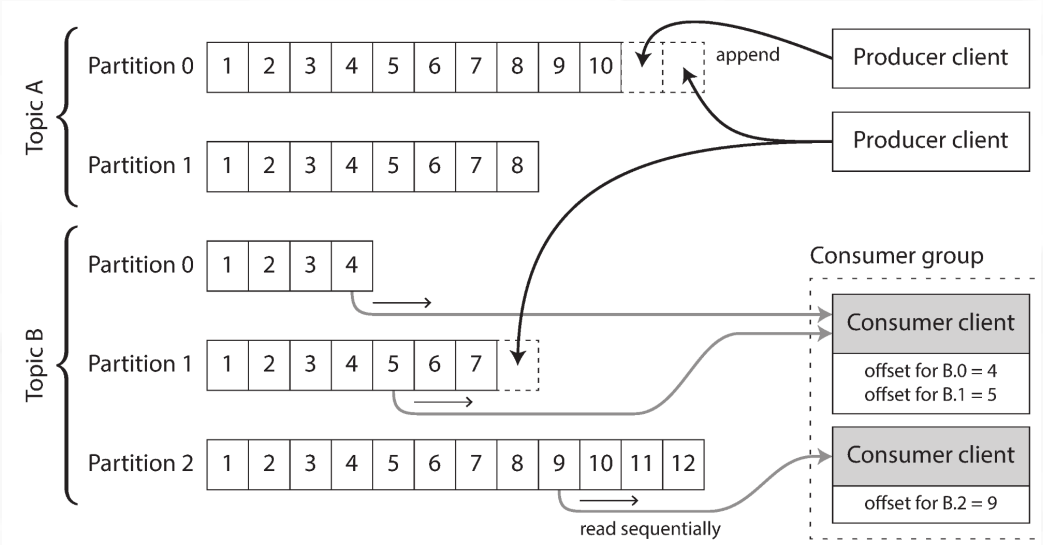
03 Partition分区

- Kafka 中 Topic 被分成多个 Partition 分区。
- Topic 是一个逻辑概念，Partition 是最小的存储单元，掌握着一个 Topic 的部分数据。
- 每个 Partition 都是一个单独的 log 文件，每条记录都以追加（append）的形式写入。
- Kafka 把 Topic 的多个 Partition 分布在多个 Broker 中



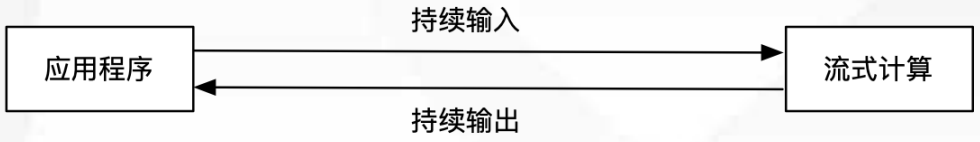
03 Offsets和消息的顺序

- Offset
  - Partition 中的每条记录都会被分配一个唯一的序号，称为 **Offset**（偏移量）。
  - Offset 是一个递增的、不可变的数字，由 Kafka 自动维护。
  - 当一条记录写入 Partition 的时候，它就被追加到 log 文件的末尾，并被分配一个序号，作为 Offset
- 消息的顺序
  - 一个 Partition 内部消息有序，一个 Topic 跨 Partition 是无序的。
  - 如果强制要求 Topic 整体有序，就只能让 Topic 只有一个 Partition。

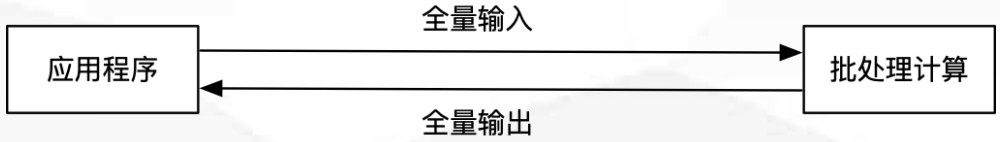


02 基于Kafka的流处理应用

- Kafka也常用于实时流处理，能够处理来自各个流的实时数据，支持数据的过滤、转换、聚合等操作。
- 结合流式计算引擎（如Apache Flink或Spark Streaming）进一步处理和分析
- 应用场景：
  - 实时分析：实时对数据流进行分析、计数、监控等。
  - 数据流转发：在数据流到达后，Kafka将其发送到目标系统进行进一步处理。



流处理应用



批处理应用

02 基于Kafka的流处理应用

西南交通大学 Southwest Jiaotong University

Kafka Streams use cases

The New York Times

[The New York Times uses Apache Kafka](#) and the Kafka Streams API to store and distribute, in real-time, published content to the various applications and systems that make it available to the readers.

[Pinterest uses Apache Kafka and the Kafka Streams API](#) at large scale to power the real-time, predictive budgeting system of their advertising infrastructure. With Kafka Streams, spend predictions are more accurate than ever.

zalando

As the leading online fashion retailer in Europe, Zalando uses Kafka as an ESB (Enterprise Service Bus), which helps us in transitioning from a monolithic to a micro services architecture. Using Kafka for processing [event streams](#) enables our technical team to do near-real time business intelligence.

LINE

LINE uses Apache Kafka as a central datahub for our services to communicate to one another. Hundreds of billions of messages are produced daily and are used to execute various business logic, threat detection, search indexing and data analysis. LINE leverages Kafka Streams to reliably transform and filter topics enabling sub topics consumers can efficiently consume, meanwhile retaining easy maintainability thanks to its sophisticated yet minimal code base.

Rabobank

Rabobank is one of the 3 largest banks in the Netherlands. Its digital nervous system, the Business Event Bus, is powered by Apache Kafka. It is used by an increasing amount of financial processes and services, one which is Rabo Alerts. This service alerts customers in real-time upon financial events and is built using Kafka Streams.

DEVSISTERS

We use Apache Kafka (and Kafka Streams) to collect and ingest all of our game service logs (including analytics, server, or access logs). Apache Kafka has been one of the core components of our data pipeline from early 2015.

ironSource

ironSource powers the growth of the world's top games, using Apache Kafka as the backbone infrastructure for the async messaging of millions of events per second that run through their industry-leading game growth platform. In addition ironSource uses the Kafka Streams API to handle multiple real-time use cases, such as budget management, monitoring and alerting.

kpow

Kpow is an enterprise-grade toolkit that provides a rich, data-oriented UI and secure API for Apache Kafka. It's designed to give engineers deep visibility and control over their Kafka clusters, Schema Registry, Kafka Connect, and Kafka Streams applications. Offering multiple deployment options for cloud or on-premise environments, Kpow works with any Kafka distribution and integrates with enterprise security systems for authentication and

西南交通大学
Southwest Jiaotong University

3 RabbitMQ

01 AMQP协议基础

- **AMQP（Advanced Message Queuing Protocol）**是一个广泛应用于消息队列中的开放标准协议。
- 目标：为消息传递提供标准化的方式，确保消息系统之间的互操作性。
- 关键特性：
  - **可靠性**：确保消息在传递过程中不丢失。
  - **可扩展性**：支持从小型系统到大型分布式系统的扩展。
  - **灵活性**：支持多种消息传输模式，包括点对点、发布/订阅等。

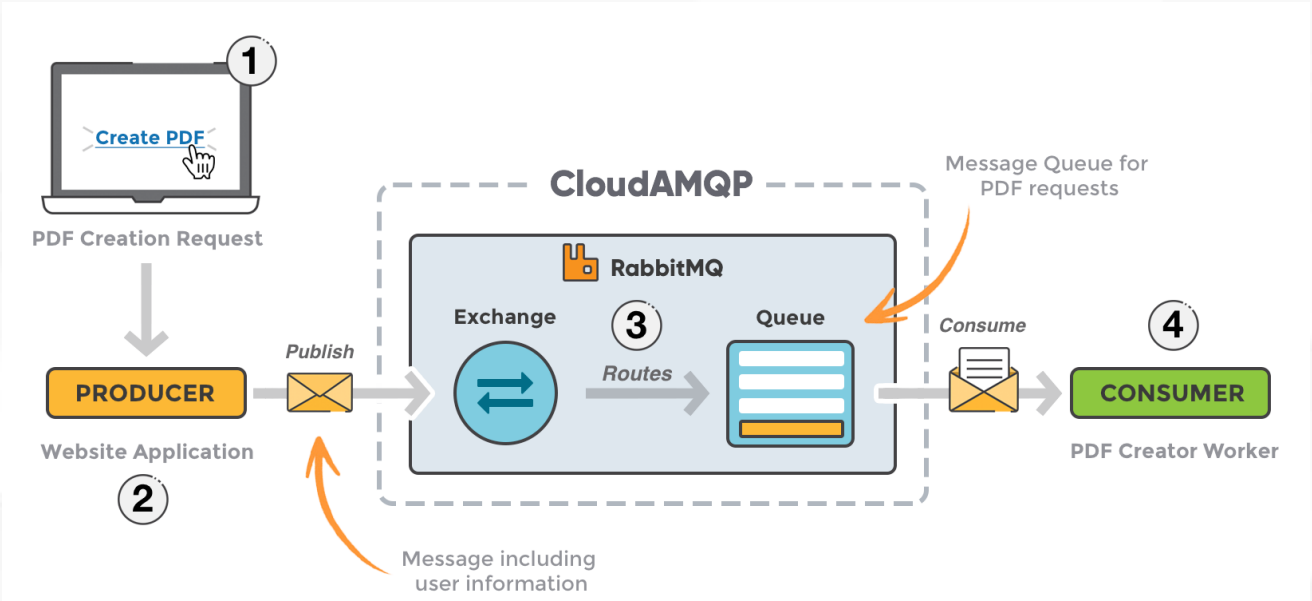
02 RabbitMQ核心概念

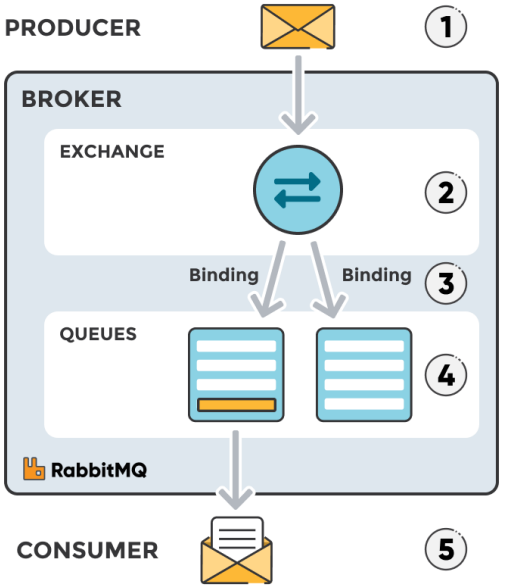
西南交通大学 Southwest Jiaotong University

- **RabbitMQ是基于AMQP协议实现的消息队列系统**
- RabbitMQ如何实现AMQP协议：
  - 通过AMQP的定义来管理Producer、Consumer和消息的传递。
  - 提供AMQP协议中的核心功能：Exchange、Queue、Binding、Routing Key等。
  - 提供附加功能（如消息确认机制、死信队列、延迟消息等）来增强其功能。

02 RabbitMQ核心概念

概念	说明
Exchange	负责接收消息并将其路由到适当的队列。
Queue	存储消息，消费者从中获取消息进行处理。
Binding	将Exchange与Queue绑定的规则，定义消息路由路径。
Routing Key	消息的路由关键字，用于匹配Exchange与Queue的绑定。





03 RabbitMQ中Exchange类型

西南交通大学 Southwest Jiaotong University

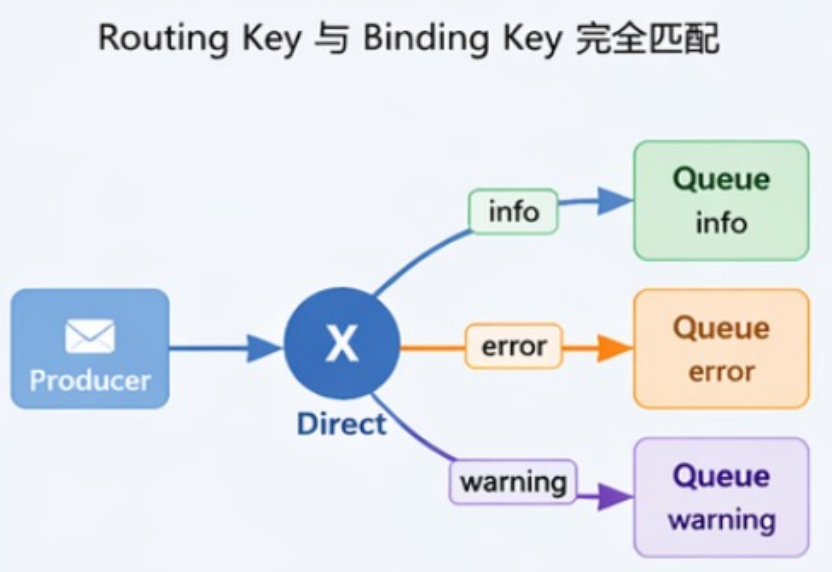
生产者把消息发给 Exchange 后，RabbitMQ 到底是怎么决定“把消息送到哪个队列”？

- **Exchange 的作用**：接收生产者发送的消息，并按照规则路由到一个或多个队列
- **为什么不直接发给 Queue?**
  - 生产者不应该关心具体消费队列
  - 便于解耦、扩展和灵活路由
- **不同业务需要不同路由方式**
  - 精确投递 Direct
  - 广播投递 Fanout
  - 按模式匹配投递 Topic

03 Direct Exchange

西南交通大学 Southwest Jiaotong University

- **路由规则**：Routing Key（消息携带）与 Binding Key（队列设置） **完全匹配**
- **特点**
  - 路由简单
  - 适合精确分类投递
- **适用场景**
  - 日志等级分发：info / warning / error
  - 订单按业务类型分发
  - 不同任务发送到不同处理队列



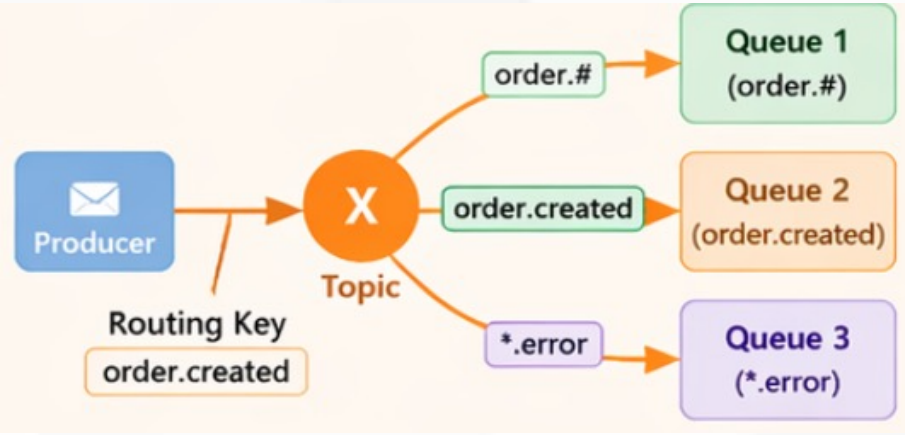
03 Fanout Exchange

西南交通大学 Southwest Jiaotong University

- **路由规则**：忽略 Routing Key，把消息**广播发送**到所有绑定队列
- **特点**
  - 一条消息可被多个消费者同时接收
  - 适合广播通知
- **适用场景**
  - 系统公告
  - 配置更新通知
  - 聊天室群发消息
  - 缓存失效广播

03 Topic Exchange

- **路由规则**：Routing Key 与 Binding Key按**模式匹配**
- **常用通配符**：
  - *：匹配一个单词
  - #：匹配零个或多个单词
- **示例**
  - order.created
  - order.paid
  - user.login
- **适用场景**
  - 事件分类分发
  - 微服务事件总线
  - 日志按模块与级别分发



Topic Exchange 特别适合事件驱动架构。
事件天然就适合按照“领域.动作”这种方式命名

绑定键	含义
order.*	匹配 order.created、order.paid
*.error	匹配 db.error、app.error
order.#	匹配所有 order 开头的主题

03 RabbitMQ中Exchange类型

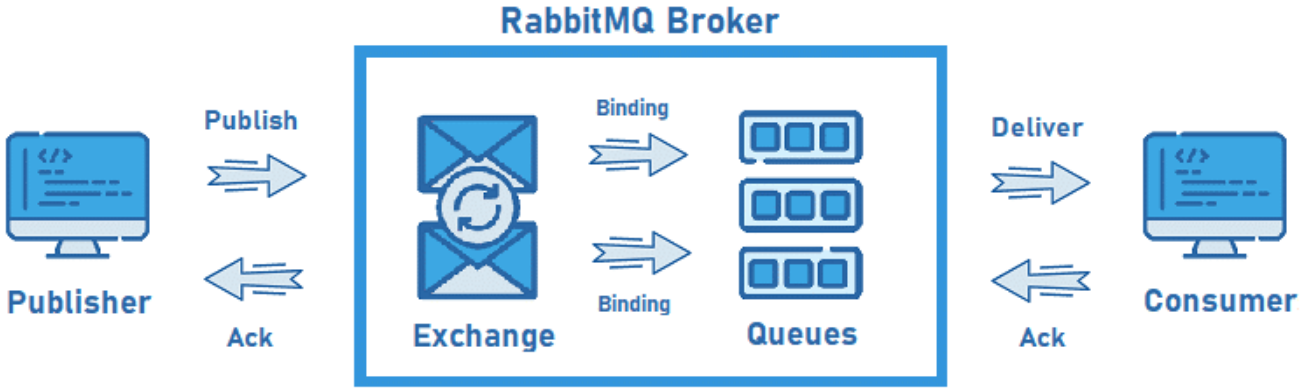
西南交通大学 Southwest Jiaotong University

类型	路由方式	是否支持一对多	是否依赖 Routing Key	适用场景
Direct	完全匹配	可有限支持	是	精确分类、任务分发
Fanout	广播到所有绑定队列	是	否	广播通知、群发
Topic	模式匹配	是	是	事件总线、主题订阅

03 消息确认机制（ACK）

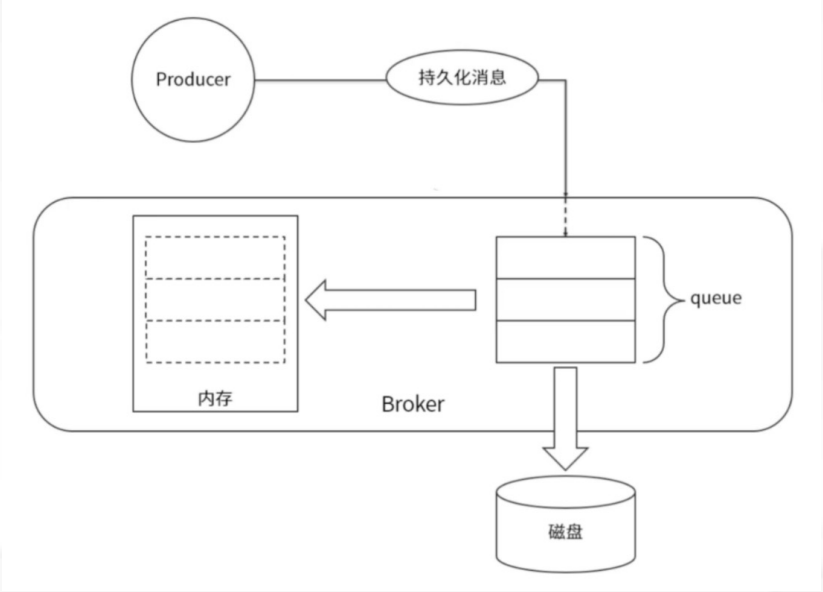
西南交通大学 Southwest Jiaotong University

- 消息确认机制（ACK）：
  - 消费者处理完消息后发送ACK（确认）消息给RabbitMQ，告知消息已成功处理
  - 作用：确保消息不会丢失，只有当消费者确认后，消息才被标记为已处理。

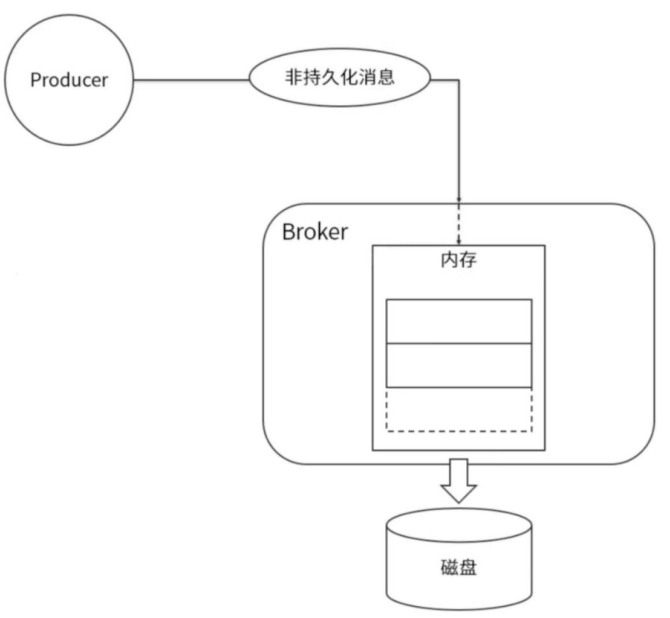


03 消息持久化

- 消息持久化：
  - 持久化消息会同时写入磁盘和内存（加快读取速度）
  - 非持久化消息会在内存不够用时，将消息写入磁盘（一般重启之后就没有了）
  - 作用：增强RabbitMQ的可靠性和持久性



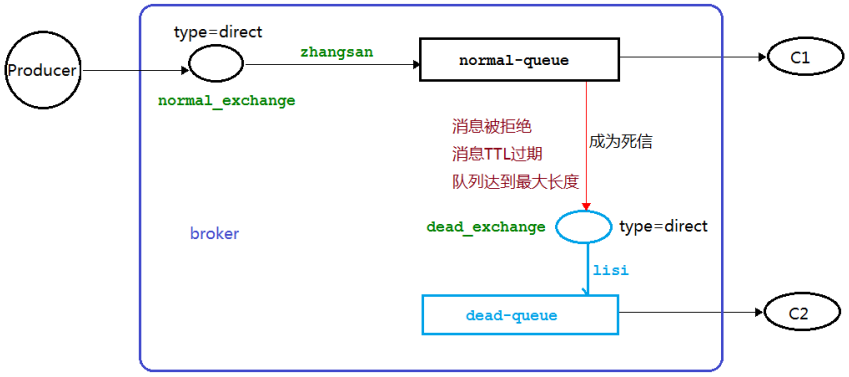
持久化



非持久化

04 死信队列（DLQ）与延迟消息

- **死信队列（Dead Letter Queue）**：当消息无法被正常消费时，消息将被转移到的专门队列
  - **死信（Dead Letter）**：无法被正常消费的消息
  - 例如：用户在商城下单成功并点击去支付后在指定时间未支付时自动失效
- 死信发生的常见原因：
  - 消息 TTL （Time To Live）过期
  - 队列长度达到最大值，消息被丢弃
  - 消息被拒绝
- **延迟消息**
  - 通过设置消息的延迟时间（如消息处理失败后，隔段时间再消费），消息会在指定的时间后再被消费。
  - 实现方式：延迟交换机（Delayed Message Exchange）来指定消息的延迟时间

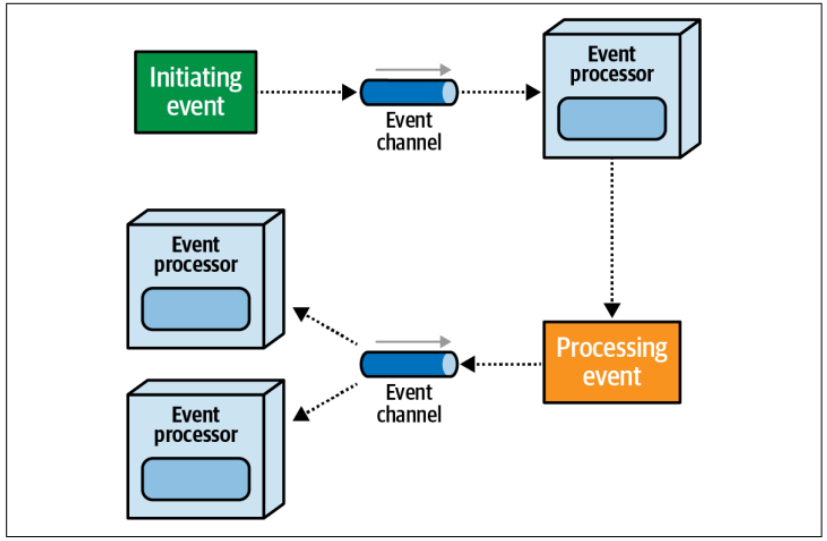


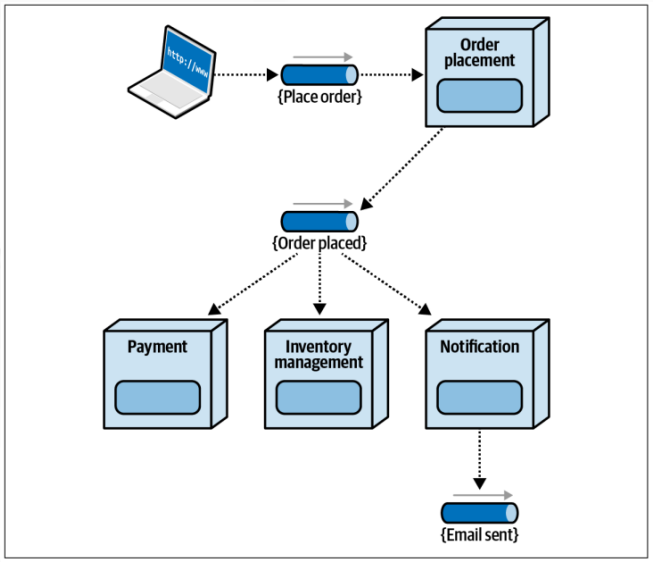
西南交通大学
Southwest Jiaotong University

4 事件驱动架构（EDA）

01 事件驱动架构（EDA）概述

- **事件驱动架构（EDA）** 是一种基于**异步处理**的架构，通过**事件**来驱动系统的行为，而非通过请求和响应
  - **松耦合**：各组件通过事件进行解耦，不直接依赖彼此的实现。
  - **异步处理**：事件发布和消费是异步的，可以有效提高系统的响应能力和处理性能。
  - **高可扩展性**：易于扩展和维护，适应复杂的分布式系统。





02 事件驱动架构V.S. 请求/响应模式

特性	事件驱动架构（EDA）	请求/响应模式
通信方式	通过事件传递，发布者与消费者异步解耦	客户端发送请求，服务器同步响应
系统耦合性	松耦合，服务之间通过事件进行解耦	紧耦合，客户端和服务器直接依赖
响应时间	异步处理，响应时间不确定	同步处理，响应时间明确
扩展性	易于扩展，能够轻松增加新的事件与服务	扩展性较差，需要修改请求与响应的处理逻辑
适用场景	实时数据处理、事件流、日志收集等	简单的客户端-服务器交互，传统Web应用
可靠性	可能存在消息丢失或延迟，但可以通过补偿机制处理	请求必须得到及时响应，否则连接超时
性能	高并发场景下，性能较好	性能受限于请求/响应的同步方式

02 事件溯源（Event Sourcing）与CQRS

西南交通大学 Southwest Jiaotong University

- **事件溯源（Event Sourcing）**：通过存储**所有事件**（而不是最终的状态），来重新构建系统的状态。这些事件作为系统状态变化的记录。

- **CQRS（Command Query Responsibility Segregation）**
  - 将读取操作（Query）和写入操作（Command）分离的模式，每个操作由不同的模型或服务处理
  - 命令（Command）：事件持久化
  - 查询（Query）：事件读取，从事件序列派生。
  - 优势：通过分离读写操作，能够针对不同操作进行性能优化；读操作与写操作的处理可以独立扩展。

西南交通大学
Southwest Jiaotong University

5 Redis Pub/Sub与Stream

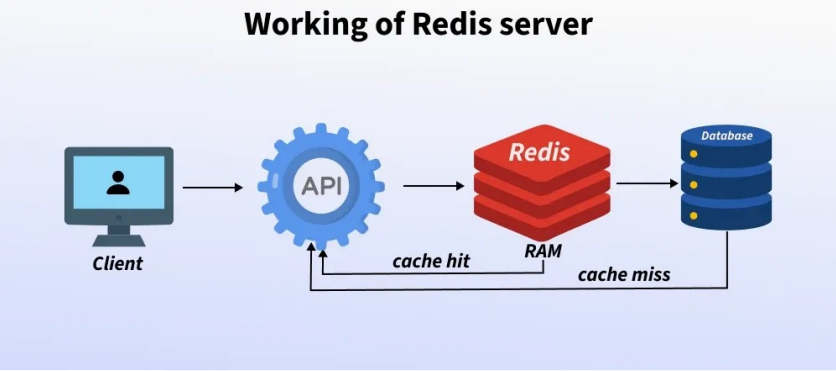
01 Redis简介

西南交通大学 Southwest Jiaotong University

当业务需要的数据增删改查速度>>数据库增删改查速度时，该怎么办?

- Redis 是一个开源的高性能键值对存储系统，通常作为数据库、缓存和消息中间件使用
- 核心思想：数据放在本地缓存（RAM）加快读写
- 应用场景：实时通信、实时监控（实时数据收集、实时日志处理等）

特性	描述
内存存储	所有数据存储在内存中，提供极低的访问延迟。
高性能	每秒处理数百万次读写操作，适合高并发应用。
持久化	支持将数据持久化到磁盘，确保数据不丢失。
简单易用	支持多种编程语言，易于部署和管理。
丰富数据结构	支持字符串、哈希、列表、集合、有序集合等多种数据类型。
多语言支持	Redis支持多种编程语言，且社区活跃，文档齐全。



01 Redis作为轻量级消息中间件的优势

西南交通大学 Southwest Jiaotong University

- **低延迟与高性能：**
  - Redis在内存中存储数据，因此其读取和写入速度极快，适合处理高并发请求。
  - 适用于需要快速响应的场景，如实时消息传递、状态更新等。
- **轻量级设计：**
  - Redis是一个简单的键值存储，不需要复杂的配置或依赖，易于部署和扩展。
  - 可以作为临时存储中间件，在多种应用中作为消息队列进行快速数据交换。
- **高吞吐量：**
  - Redis可以处理每秒数百万条消息，非常适合高并发的系统和流量较大的应用。

```python
import redis
r = redis.Redis(host='localhost', port=6379, db=0)

r.set('name', 'Alia')
print(r.get('name').decode('utf-8'))

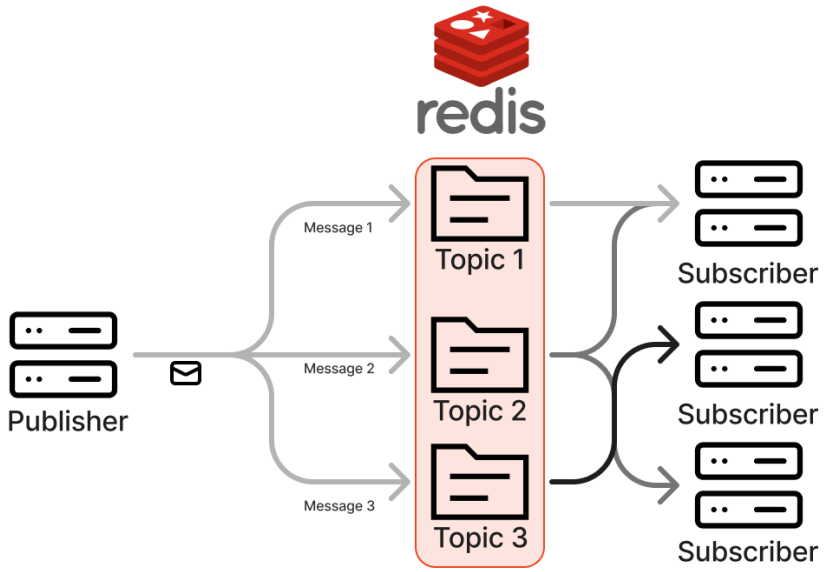
r.set('name', 'Riya')
print(r.get('name').decode('utf-8'))

r.delete('name')
print(r.get('name'))
```

```
(.venv) gfg0622@Anshu-ka-lappi:~/Downloads$ /
Alia
Riya
None
(.venv) gfg0622@Anshu-ka-lappi:~/Downloads$
```

02 Pub/Sub模式

- **Pub/Sub（发布/订阅）：是一种消息通信模式，其中发布者发布消息，订阅者订阅这些消息**



优点	缺点
松耦合：发布者和订阅者之间没有直接依赖，方便扩展系统。	**消息丢失**：如果订阅者离线，消息可能会丢失。
实时性：消息实时推送，适用于需要即时响应的应用。	**不保证消息顺序**：不同消费者处理顺序可能不同。
多对多通信：支持多个发布者和多个订阅者的多对多通信模式。	**难以跟踪消息**：无法直接跟踪消息的处理状态。

03 Redis Stream

西南交通大学 Southwest Jiaotong University

- Redis Stream是Redis的一个新特性，它允许**持久化存储消息**，并可以对消息进行消费、确认和追踪。
- 持久化消息队列：
  - 消息可以持久化到磁盘中，即使Redis重启，数据不会丢失
  - 支持消息的按时间排序，确保有序消费。

特性	Stream	Pub/Sub
数据模型	基于消息的流，支持持久化和消费记录	发布者发布消息，订阅者接收消息
持久化	支持持久化（存储到RDB或AOF文件）	不支持持久化，消息丢失后无法恢复
消息确认	支持消息确认（ACK），确保消息被消费者成功处理	不支持消息确认
消费者组支持	支持消费者组，允许多个消费者并行消费消息	不支持消费者组
顺序性保证	支持顺序消费，消息按发送顺序消费	不保证顺序性，多个订阅者消费可能导致顺序错乱
消息撤回/重试	支持从特定位置开始消费消息（通过ID），可以重试未处理的消息	不支持撤回或重试
性能	性能与消费者数量无关，适合高吞吐量场景	性能与消费者数量正相关，较多的订阅者可能导致性能下降
数据过期/清除	支持消息清除策略（例如最大长度或过期时间）	不支持数据清除
适用场景	高吞吐量、日志收集、事件驱动架构、消息队列	实时消息推送、实时通知、广播通讯
支持随机删除	不支持	不支持随机删除

西南交通大学
Southwest Jiaotong University

6 应用案例与代码演示

01 演示 1：kafka-python消息发布与分区消费

西南交通大学 Southwest Jiaotong University

- Kafka 生产者（Producer）：发布消息、指定分区

```
from kafka import KafkaProducer
import json

# 创建Kafka生产者实例，连接到Kafka服务
producer = KafkaProducer(
    bootstrap_servers=['localhost:9093'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')  # 将消息序列化为JSON格式
)

# 发送消息到指定的Topic，并指定分区（比如分区0）
producer.send('test-topic', value={'key': 'value'}, partition=0)

# 关闭生产者
producer.flush()
producer.close()

print("Message sent to Kafka!")
```

01 演示 1：kafka-python消息发布与分区消费

西南交通大学 Southwest Jiaotong University

- Kafka消费者
  - 获取消息
  - 从 test-topic 中消费消息，并实现分区消费

```
from kafka import KafkaConsumer
import json

# 创建Kafka消费者实例，连接到Kafka服务，订阅指定Topic
consumer = KafkaConsumer(
    'test-topic',
    bootstrap_servers=['localhost:9093'],
    group_id='test-consumer-group',  # 消费者组
    auto_offset_reset='earliest',  # 从最早的消息开始消费
    value_deserializer=lambda x: json.loads(x.decode('utf-8'))  # 反序列化消息
)

# 逐个获取消息并打印
for message in consumer:
    print(f"Consumed message: {message.value} from partition: {message.partition}")
```

02 演示 2：Pub/Sub实时消息通信

西南交通大学 Southwest Jiaotong University

- 安装 redis-py 库　　**pip install redis**
- 发布者将消息发布到一个指定的频道，所有订阅该频道的消费者将接收到消息

```
import redis
import time


# 连接到Redis服务器
client = redis.StrictRedis(host='localhost', port=6379, db=0)


# 发布消息到 'chat_channel'
def publisher():
    while True:
        message = input("Enter a message: ")
        client.publish('chat_channel', message)  # 发布消息到 'chat_channel'
        print(f" [x] Sent: {message}")
        time.sleep(1)  # 每隔1秒发布一条消息


# 启动发布者
publisher()
```

06 演示 2：Pub/Sub实时消息通信

西南交通大学 Southwest Jiaotong University

- 订阅者订阅 chat_channel，并在接收到消息时进行处理

```
import redis

# 连接到Redis服务器
client = redis.StrictRedis(host='localhost', port=6379, db=0)

# 订阅 'chat_channel' 频道
def subscriber():
    pubsub = client.pubsub()  # 创建pubsub对象
    pubsub.subscribe('chat_channel')  # 订阅频道


    print(" [*] Waiting for messages. To exit press CTRL+C")
    for message in pubsub.listen():  # 监听频道
        if message['type'] == 'message':  # 如果收到消息
            print(f" [x] Received: {message['data'].decode()}")

# 启动订阅者
subscriber()
```

03 课堂思考

西南交通大学 Southwest Jiaotong University

- 思考题
  - 消息队列能如何解决系统间的耦合问题?
  - 在选择消息投递语义时，如何根据不同的应用场景（如支付系统与日志收集）选择合适的消息投递策略?
  - Kafka中的单分区顺序性保证和多分区顺序性不保证有何区别？在实际应用中，如何选择分区数目以平衡性能和消息顺序性?
  - 事件驱动架构（EDA）在微服务设计中的优势是什么？如何通过事件溯源和CQRS模式提高系统的扩展性和可靠性?